Department of Computer Science and Engineering

PROJECT REPORT

ON

" Hospital Management System Unit Testing"

Course Name: Software Testing and Quality Assurance

Course Code: CSE430

Section No: 01

Semester: Spring 2025

Submitted By:

Group No	Name	ID
02	**Rifa Azad**	**2022-1-60-265**
	Tahura Akter Tripty	**2022-1-60-051**
	Shaiful Islam	**2022-1-60-120**
	Umme Habiba Rupu	**2022-1-60-334**
	Tasnuva Islam Mila	**2022-1-60-033**

Submitted To:

Anika Tabassum

Lecturer

Department of Computer Science and Engineering

Date of Submission: 22-05-2025

Abstract
The Hospital Management System is a Java-based application that enables efficient management of appointments, patients, and medical staff. To ensure the reliability and correctness of the system, JUnit testing was employed. This report explores the approach taken to test the system's functionality, the methodology used to perform tests, and the results from running unit tests on appropriate classes.

Introduction
The Hospital Management System (HMS) is a Java-based application that manages hospital operations, including appointments, doctor and patient records, and validation of user inputs. There are about **32 classes** among which **3 classes are interfaces**, **3 classes are Abstracts**. We have tested **24 classes** (excluding interfaces, menu/UI and runner class) and cover the possible cases of those functions while maintaining the dependency.
To assure the quality of the project's test we have used **JUnit5** test library **in Intellij IDE environment.**

Purpose of Testing
The purpose of conducting JUnit tests on the Hospital Management System is to ensure the system's key components function as intended. These tests are designed to validate the correctness, reliability, and robustness of the core business logic, including appointment scheduling, patient and doctor management, and data validation processes.
Ultimately, these tests help improve the overall quality and maintainability of the Hospital Management System, ensuring it meets functional requirements and operates without unexpected failures.

Test Classes Overview:

Testing class	Testing functions
BranchesTest.java	constructor_Default(), constructor_IdAndBranch(), testConstructor_IdBranchDoctors(), constructor_IdException(), constructor_nameClash_Exception(), SetId_ValidValue(), SetId_Negative(), setId_DuplicateId(), testToString(), getDoctorsList_ReturnsCopy(), setDoctorsList_ValidList(), setDoctorsList_Null(), fillBranchList(), setBranch_Valid(), setBranch_EmptyString(), setBranch_WhiteSpace(), getBranchById_Valid(), getBranchById_Invalid(), getBranchById_NegativeId(), addBranch_DuplicateID(), getAllBranchesReturnsCopy(), addBranch_NewBranch(), addBranch_Null(), addBranch_WhitespaceName(), removeBranch_Existing(), removeBranch_NonExisting(), addBranch_EmptyBranchName(), addBranch_NegativeID(), branchRemoval_RemovesDoctorReferences()
EntityBaseTest.java	singleConstructor_SetorNot(), singleConstructor_TimeRange(), constructor_AllFields(), constructor_AllFieldsNullAndFalse(), setId_ValidID(), setId_ThrowExceptionForNull(), setId_ThrowExceptionForEmptyString(), setId_ThrowExceptionForWhitespaceOnly(), setCreatedDate_DefaultReturn(), setCreatedDate_FutureDate(), setCreatedDate_PastDate(), setCreatedDate_NullDate(), setModifiedDate_ReturnSetValue(), setActiveTest(),

	setModifiedDate_FutureDate(), setModifiedDate_PastDate(), setDeletedTest(), setModifiedDate_AcceptNull(), setCreatedByName_NormalString(), setCreatedByName_SpecialChar(), setCreatedByName_Numbers(), setCreatedByName_EmptyString(), setCreatedByName_Null(), modifiedByName_NormalString(), setModifiedByName_SpecialCharacters(), setModifiedByName_Numbers(), setModifiedByName_EmptyString(), setModifiedByName_Null(), isDeletedReturnDefault(), isActiveReturnDefault()
DoctorsTest.java	defaultConstructor(), constructorWith_UserDetails(), constructorWithoutUserDetails(), setTitle_NullInput(), setTitle_Valid(), setBranch_Valid(), setBranch_Null(), setDoctorSituation_Valid(), setDoctorSituation_Null(), addDoctor_Valid(), addDoctor_Null(), addDoctor_DuplicateId(), removeDoctor_ExistingId(), testToString(), removeDoctor_NonExistingId(), fillDoctorList_valid(), fillDoctorList_DuplicacyTest(), testShowUsers(), getDoctorById_ExistingId(), getDoctorById_NonExistingId(), getDoctorsList(), getDoctorsList_CopyOfList(), setFirstName_InheritedMethod(), setLastName__InheritedMethod()
PatientsTest.java	defaultConstructor(), parameterizedConstructor(), minimalParameterConstructor(), setTcNo_ValidInput(), setTcNo_NullInput(), setTcNo_InvalidLength(), testSetTcNo_InvalidChar(), setBirthDate_ValidInput(), setBirthDate_NullInput(), setBirthDate_FutureDate(), setBirthDate_PresentNowDate(), setGender_Valid(), setGender_NullInput(), testSetPriority(), setComplaintTest(), setLastStatus(), testToString(), fillPatientsListTest(), noDuplicacyInMultipleFill(), getPatientById_NonExistingId(), getPatientById_ExistingId(), getPatientById_NullId(), addPatient_NonNull(), addPatient_NullPatient(), addPatient_DuplicateId(), removePatient_ExistingId(), removePatient_NonExistingId(), getAllPatientsTest(), getAllPatients_CopyOfList(), showUsers_PrintPatientList(), setFirstName_InheritedMethod(), setLastName__InheritedMethod()
ComplaintsTest.java	defaultConstructorSuccessfull(), constructorWithIdComplaintAndPatientsList(), constructorWithIdAndComplaint(), constructorWithNullPatientsList(), setId_valid(), setId_MIN(), setId_MAX(), setComplaint_Valid(), testNullComplaintString(), setPatientsListValid(), setPatientsList_NullList(), testToString(), setPatientsList_OriginalListUnmodified(), complaintsListInitialization(), fillComplaintListNoDuplicates(), getComplaintById_Valid(), getComplaintById_NonExistingComplain(), patientComplaintRelationship(), multiplePatientsWithSameComplaint(), multipleComplaintBYOnePatient()
AppointmentTest.java	constructorWithAutoGeneratedId(), fullConstructor(), constructorSuperClass(), testCountByCreatingAppointments(), setCountTest(), testIdGeneration(), testIdGenerationFormat(), setPatientValid(), setPatientNullException(), setDoctorValid(), setDoctor_Null(), setAppointmentDateNullException(), setAppointmentDateValid(), setAppointmentDateToday(), setAppointmentDate_PastDateException(), setAppointmentTimeValid(), setAppointmentTime_RejectNUll(), getStatusReturnsDefaultEnumStatus(), setAppointmentTimeOutsideBusinessHoursEarlyException(), testToString(), setAppointmentTimeOutsideBusinessHoursLateException(), setStatusTest(), testMultipleAppointmentsWithSameDoctorDifferentTime(), testSameDoctorSameTimeDifferentPatientsShouldNotBeAllowed(), testSamePatientDifferentDoctorsSameTimeShouldNotBeAllowed()
UsersTest.java	testSetterWithValidId(), testRepeatedSetters(), testValidUserCreation(), testUpdateToNewId(), testSetIdNull(), testSetIdEmpty(),

	testSetIdSpecialCharacter(), testSetIdWithSpace(), testSetIdCaseSensitive(), testSetIdToDuplicate(), testValidFirstName(), testRepeatedSettersFirstName(), testFirstNameCaseSensitive(), testUpdateToNewFirstName(), testSetFirstNameNull(), testSetFirstNameEmpty(), testSetFirstNameWithDigit(), testSetFirstNameWithSpecialCharacter(), testSetFirstNameWithSpace(), testDuplicateFirstNameForDifferentId(), testValidLastName(), testRepeatedSettersLastName(), testLastNameCaseSensitive(), testUpdateToNewLastName(), testSetLastNameNull(), testSetLastNameEmpty(), testSetLastNameWithSpecialCharacter(), testSetLastNameWithDigit(), testSetLastNameWithSpace(), testDuplicateLastNameForDifferentId(), testShowUsersDoesNotThrow()
StartTest.java	testExitOption(),testInvalidIntegerInput(),testNonIntegerInput(),testNonDigitInput(),testNegativInput(),testLargeInputNumber(),testSpecialCharacterInput(),testInputNull()
AmbulanceScheduleValidatorTest.java	testValidPickupDate(), testPastPickupDate(), testInvalidDayPickupInput(), testInvalidMonthPickupInput(), testNonNumericPickupInput(), testNegativePickupDate(), testLeapYearPickupDate(), testLeapYearReturnDate(), testInvalidLeapYearPickupDate(), testValidReturnDate(), testInvalidReturnDate(), samePickupReturnDate(), testInvalidDayReturnDate(), testInvalidMonthReturnDate(), testNonNumericReturnDate(), testNegativeReturnDate(), testInvalidLeapYearReturnDate()
TimeValidatorTest.java	testValidInput(), testInvalidInput(), testZeroHourZeroMinute(), testUpperBoundaryTime(), testOutOfBoundHour(), testOutOfBoundMinute(), testNegativeHour(), testNegativeMinute()
TcNoValidatorTest.java	testValidTcNumber(),testNonDigitInput(),testShortTcNumber(),testLongTcNumber(),testNegativeInput(),testSpecialCharacterInput(),testNullInput(),testWhiteSpaceTcNumber()
NameValidator.java	isValidFirstName(), isValidFirstName_allLowerCase(), isValidFirstName_middleUpperCase(), isValidFirstName_multipleUpperCase(), isValidFirstName_WithEmptyInput(), isValidFirstName_WithOnlySpaces(), isValidFirstName_startWithSpaces(), isValidFirstName_endWithSpaces(), isValidFirstName_startAndEndWithSpaces(), isValidFirstName_endWithNumbers(), isValidFirstName_startWithNumbers(), isValidFirstName_startAndEndWithNumbers(), isValidFirstName_mixedNumbers(), isValidFirstNameWithSpecialInput1(), isValidFirstNameWithSpecialInput2(), isValidFirstNameWithSpecialInput3(), isValidLastName(), isValidLastName_allLowerCase(), isValidLastName_middleUpperCase(), isValidLastName_multipleUpperCase(), isValidLastName_WithEmptyInput(), isValidLastName_WithOnlySpaces(), isValidLastName_startWithSpaces(), isValidLastName_endWithSpaces() , isValidLastName_startAndEndWithSpaces(), isValidLastName_endWithNumbers(), isValidLastName_startWithNumbers(), isValidLastName_mixedNumbers(), isValidLastName_startAndEndWithNumbers(), isValidLastName_SpecialChars1(), isValidLastName_SpecialChars2(), isValidLastName_SpecialChars3(), isValidLastName_SpecialChars3()
Gender.java	male(), female(), other(), zero(), negative(), largePositive(), outOfRangeInput(), characterInput()
DoctorSituation.java	getId(), getId_notEquals(), getSituation(), getSituation_notEquals1(), fillSituation_listSize(), fillSituation_getFirstIndexID(), fillSituation_getFirstIndexSituation(), fillSituation_getThirdIndexSituation(), fillSituation_getForthIndexID(), fillSituation_getForthIndexSituation(), fillSituation_getIdNotEquals(), fillSituation_getSituationNotEquals(), getSituationById1(), getSituationById_outOfList(), getSituationById_negativeId()

LastStatus.java	getId(), getId_notEquals(), getLastStatus(), getLastStatus_notEquals(), fillLastStatusList_getListSize(), fillLastStatusList_getFirstIndexSituation1(), fillLastStatusList_getFirstIndexSituation2(), getLastStatusById(), getLastStatusById_outOfList(), getLastStatusById_zero(), getLastStatusById_negativeIndex()
Priorities.java	getId(), getId_notEquals(), getPriority(), getPriority_notEquals(), fillPrioritiesList_getListSize(), fillPrioritiesList_firstIndexPriority(), fillPrioritiesList_notEquals(), getPrioritiesById(), getPrioritiesById_outOfIndex(), getPrioritiesById_zeroIndex(), getPrioritiesById_negativeIndex()
Titles.java	getId(), getId_notEquals(), getTitle1(), getTitle_notEquals(), testToString1(), fillTitles_getListSize(), fillTitles_getFirstIndexTitle(), testGetTitleById_zeroIndex(), fillTitles_getFirstIndexTitleNotEquals(), getTitleById(), testGetTitleById_outOfIndex(), testGetTitleById_negativeIndex()
Appointment Manager.java	testIdMaker_withValidNumber(),testIdMaker_withNullNumber(),estIdMaker_withEmptyString(),testIdMaker_withInvalidCharacters(),testIdMaker_withSpecialCharacter()
DoctorManager.java	idMaker_WithShortNumber(),idMaker_WithLongNumber(),idMaker_LessThanThreeDigits(),idMaker_NullInput(),idMaker_EmptyString().
PatientManager.java	idMaker_shouldGenerateCorrectIdWithLongNumber(),idMaker_shouldGenerateCorrectIdWithShortNumber(),idMaker_shouldThrowExceptionForLessThanThreeDigits(),idMaker_shouldHandleNullInputGracefully(),idMaker_shouldHandleEmptyString
DateValidatorTest.java	testValidDateInput(),testInvalidThenValidDateInput(),testInvalidDateThenValid(), testLeapYearValidDate(),testLeapYearInvalidDateThenValid(),testInvalidDateFormat()
IdMakerTest	idMaker_shouldPrefixWithID(),idMaker_shouldWorkWithEmptyString(),idMaker_shouldWorkWithAlphabets(),idMaker_shouldThrowForSpecialCharacters(),idMaker_shouldWorkWithSpaces(),idMaker_shouldThrowNullPointerExceptionForNullInput(),idMaker_shouldWorkWithLongInput(),idMaker_shouldBeCaseSensitive()
InputValidatorTest	testValidNumericInput(), testInvalidNonNumericInput(),testEmptyInput(), testInputWithSpaces(),testMixedInput(),testMultipleInvalidInputs(),testValidInputAfterEmptyAndInvalidInputs()
UtilitiesTest	testValidDoctorId_D001(),testValidDoctorId_D002(),testDoctorWithSingleSituation(), testUnknownDoctorIdreturnsEmptyList(),testEmptyDoctorIdreturnsEmptyList(),testNullDoctorIdthrowsException(),testDoctorIdIsCaseSensitive()
Excluded from Unit Testing	Runner.java, Slow.java, PatientMenuManagerTest.java, AppointmentService.java, DoctorMenuManagerTest.java, AppointmentMenuManager.java, MenuManager.java, MenuService.java

Test Cases on Each Applicable classes

Class name - BranchesTest.java

Constructors				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
constructor_Default()	1	No input	Empty Doctors list, branch name null, ID 0	✅ Pass
constructor_IdAndBranch()	1	ID: 201, Branch: "Cardiology"	ID set to 201, branch name set to "Cardiology", doctors list empty	ID set to 201, branch name set to "Cardiology", doctors list empty
constructor_IdBranchDoctors()	1	ID: 202, Branch: "Testology", Doctors list with 2 doctors	ID set to 202, branch name set to "Testology", doctors list size = 2	ID = 202, branch = "Testology", doctors list size = 2
	2	ID: 204, Branch: "NullDoctorTest", null Doctors list	ID set to 204, branch name set to "NullDoctorTest", doctors list size = 0	ID = 204, branch = "NullDoctorTest", doctors list size = 0
constructor_IdException()	1	ID: 101 (existing), Branch: "ConstructorBranchID"	Throws Exception saying "Branch ID already exists"	✅ "Branch ID already exists
constructor_nameClash_Exception()	1	ID: 777, Branch: "Psychology" (existing)	Throws Exception saying "Branch Name already exists"	✅ "Branch Name already exists"

setId(int id)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
SetId_ValidValue()	1	ID: 300	Branch ID set to 300	Branch ID = 300
SetId_Negative()	1	ID: -1	Throws IllegalArgumentException	✅ Pass
setId_DuplicateId()	1	ID: 110 (existing)	Throws Exception with message "Branch ID already exists"	✅ "Branch ID already exists

setDoctorsList(List<Doctors> doctorsList)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
getDoctorsList_ReturnsCopy()	1	Branch initialized with 2 doctors, then retrieve and clear the list	Original list should remain intact with 2 doctors, clearing only the copy	Original list remain intact with 2 doctors
setDoctorsList_ValidList()	1	Doctors list with 2 doctors	Branch's doctors list updated with 2 doctors	Branch's doctors list updated with 2 doctors
setDoctorsList_Null()	1	Null input	Empty doctors list	Empty doctors list

toString()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testToString()	1	ID: 206, Branch: "Test Branch"	Returns formatted string: "Branches{ID =206, Branch= 'Test Branch', Doctors in Branch=0}"	"Branches{ID =206, Branch='Test Branch', Doctors in Branch=0}"

fillBranchList()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
test_FillBranchList()	1	branches populated with 20 initial branches.	List with 20 branches	List with 20 branches

setBranch(String branch)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setBranch_Valid()	1	Branch: "New Branch"	Branch name = "New Branch"	Matches with Expected Output

setBranch_EmptyString()	1	Branch: ""	Throws IllegalArgumentException	✅ Pass
setBranch_WhiteSpace()	1	Branch: " "	Throws IllegalArgumentException	✅ Pass

getBranchById(int id)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
getBranchById_Valid()	1	ID: 100 (ENT branch)	Returns branch with name "ENT (Ear Nose Throat)"	Matches with Expected Output
getBranchById_Invalid()	1	ID: 999 (non-existent)	Throws IllegalArgumentException	✅ Pass
getBranchById_NegativeId()	1	ID: -1	Throws IllegalArgumentException	✅ Pass

getAllBranches()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
getAllBranchesReturnsCopy()	1	Get list, clear it	Original static list remains unchanged with 20 branches returns a copy; modifying copy does not affect original list	Original static list remains unchanged with 20 branches

addBranch(Branches branch)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
addBranch_NewBranch()	1	ID=997, Name="Valid Branch"	Branch added to list; list size increases by 1	Branch added to list; list size increases by 1
addBranch_DuplicateID()	1	ID=105, Name="Different Name”	Throws exception saying “Branch ID exists”	Throws exception saying “Branch ID exists”

addBranch_EmptyBranchName()	1	ID=999, Name= "”	Throws exception saying “Branch name can’t be empty”	Throws exception saying “Branch name can’t be empty”
addBranch_WhitespaceName()	1	ID=998, Name=" “	Throws exception saying “Branch name can’t be empty”	Throws exception saying “Branch name can’t be empty”
addBranch_NegativeID()	1	ID= -1, Name="Invalid ”	Throws exception saying “ID cannot be negative”	Throws exception saying “ID cannot be negative”
addBranch_Null()	1	Null input	Throws NullPointerException	Throws NullPointerException

removeBranch(int id)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
removeBranch_Existing()	1	ID: 888 (existing branch)	Returns true; branch removed from list; list size decreases by 1	Returns true; branch removed from list; list size decreases by 1
removeBranch_NonExisting()	1	ID: 99999 (non-existent)	Returns false; list size unchanged	Returns false; list size unchanged
branchRemoval_RemovesDoctorReferences()	1	Create branch (ID: 305), add doctor, remove branch	Branch removed; doctor object still exists but branch reference becomes invalid	Branch removed; doctor object still exists but branch reference becomes invalid

Class name - EntityBaseTest.java

EntityBase(LocalDateTime createdDate)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
singleConstructor_SetorNot()	1	No input	createdDate- Not NULL	createdDate- Not NULL

singleConstructor_TimeRange()	1	No input	createdDate - within the expected time range	createdDate - within the expected time range

EntityBase(...){constructor with all inputs}				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
constructor_AllFields()	1	"ID1", entityDate, modifyDate, "Admin", "Editor", true, false	All fields match input values by Constructor.	All fields match input values by Constructor.
	2	("ID2", null, testCase2, null, "User", false, true)	Mixed null/non-null fields works	Mixed null/non-null fields works
constructor_AllFieldsNullAndFalse()	1	(null, null, null, null, null, false, false)	All fields are null/False.	All fields are null/False.

getId()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setId_ValidID()	1	"IdCheck1"	IdCheck1	IdCheck1
	2	"a"	a	a
	3	“ID1234567890”	ID1234567890	ID1234567890
	4	“ID&^02”	ID&^02	ID&^02
	5	"User 43"	"User43"	"User43"
	6	“ User433 ”	“User433”	“User433”
	7	" User & 43 "	"User&43"	"User&43"
	8	"User _ 43"	"User_43"	"User_43"
setId_ThrowExceptionForNull()	1	null	Throws IllegalArgumentException	Throws IllegalArgumentException
setId_ThrowExceptionForEmptyString()	1	""	IllegalArgumentException	IllegalArgumentException

setId_ThrowExceptionForWhitespaceOnly	1	" "	Throws IllegalArgumentException	Throws IllegalArgumentException

getCreatedDate()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setCreatedDate_DefaultReturn()	1	(2020, 1, 1, 12, 0)	2020-01-01T12:00	2020-01-01T12:00
	2	LocalDateTime.now()	2025-05-01T15:14:48.132633	2025-05-01T15:14:48.132633
	3	(2020, 2, 29, 10, 30) {leap Year}	2020-02-29T10:30	2020-02-29T10:30
	4	(2023,12,31,0,0)	2023-12-31T00:00	2023-12-31T00:00
setCreatedDate_FutureDate()	1	LocalDateTime.now().plusYears(1)	2026-05-01T15:11:27.886445700	2026-05-01T15:11:27.886445700
setCreatedDate_PastDate()	1	LocalDateTime.now().minusYears(1)	2024-05-01T15:12:04.254507400	2024-05-01T15:12:04.254507400
setCreatedDate_NullDate()	1	null	null	null

getModifiedDate()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setModifiedDate_ReturnSetValue()	1	(2021, 5, 5, 15, 30)	2021-05-05T15:30	2021-05-05T15:30
	2	LocalDateTime.now()	2025-05-01T15:14:48.132633	2025-05-01T15:14:48.132633
	3	(2020, 2, 29, 10, 30) {leap Year}	2020-02-29T10:30	2020-02-29T10:30
	4	(2023,12,31,0,0)	2023-12-31T00:00	2023-12-31T00:00

setModifiedDate_FutureDate()	1	LocalDateTime.now().plusYears(1)	2026-05-01T15:32:11.897971100	2026-05-01T15:32:11.897971100
setModifiedDate_PastDate()	1	LocalDateTime.now().minusYears(1)	2024-05-01T15:33:04.695754600	2024-05-01T15:33:04.695754600
setModifiedDate_AcceptNull()	1	null	null	null

getCreatedByName()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setCreatedByName_NormalString()	1	"Rifa Azad"	"Rifa Azad"	"Rifa Azad"
	2	"Rifa"	"Rifa"	"Rifa"
	3	" Rifa Azad "	"Rifa Azad"	"Rifa Azad"
	4	" Han Lee Mun "	"Han Lee Mun"	"Han Lee Mun"
	5	" Han Lee Mun "	"Han Lee Mun"	"Han Lee Mun"
setCreatedByName_SpecialChar()	1	"Rifa^&"	Throws Exception	Throws Exception
	2	"r#ur"	Throws Exception	Throws Exception
	3	"#$%Rifa^&"	Throws Exception	Throws Exception
	4	"45Ri$%#fa123"	Throws Exception	Throws Exception
setCreatedByName_Numbers()	1	"Rifa123"	Throws Exception	Throws Exception
	2	"45Rifa123"	Throws Exception	Throws Exception
	3	"45Ri$%#fa123"	Throws Exception	Throws Exception
setCreatedByName_EmptyString()	1	“ “	Throws Exception	Throws Exception
setCreatedByName_Null()	1	null	Throws Exception	Throws Exception

getModifiedByName()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
modifiedByName_NormalString()	1	"poppy"	poppy	poppy
	2	" Poppy Seed "	"Poppy Seed"	"Poppy Seed"
	3	" Poppy Lee Seed "	Poppy Lee Seed	Poppy Lee Seed
	4	" Poppy Lee Seed "	Poppy Lee Seed	Poppy Lee Seed
setModifiedByName_SpecialCharacters()	1	"Poppy#Seed"	Throws Exception, Returns Default name	Throws Exception, Returns Default name
	2	"Poppy#"	Throws Exception, Returns Default name	Throws Exception, Returns Default name
	3	"#Seed"	Throws Exception, Returns Default name	Throws Exception, Returns Default name
setModifiedByName_Numbers()	1	"Poppy2"	Throws Exception, Returns Default name	Throws Exception, Returns Default name
	2	"78Poppy2"	Throws Exception, Returns Default name	Throws Exception, Returns Default name
setModifiedByName_EmptyString()	1	“ “	Throws Exception, Returns Default name	Throws Exception, Returns Default name
setModifiedByName_Null()	1	null	Throws Exception, Returns Default name	Throws Exception, Returns Default name

isDeleted()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
isDeletedReturnDefault()	1	No input	Constructor value	Constructor value
setDeletedTest()	1	true	true	true

isActive()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setActiveTest()	1	No input	Constructor value	Constructor value
isActiveReturnDefault()	1	false	false	false

Class name - ComplainTest.java

★ `patient1, patient2` - objects were created in `setup()` function and added in `testPatients` arraylist; a `Complaints` object named `complaint`.

CONSTRUCTOR TESTS				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
defaultConstructorSuccessfull()	1	No parameters	Object created successfully with empty patients list & ID set to 0	Object created successfully with empty patients list & ID set to 0
constructorWithIdComplaintAndPatientsList()	1	ID: 1111, Complaint: "Test Complaint 2", Patients list with 2 patients	Object created with specified ID, complaint, and patients list containing 2 patients	Object created with ID= 1111, complaint= "Test Complaint 2", patients list containing 2 patients
constructorWithIdAndComplaint()	1	ID: 100, Complaint: "Test Complaint"	Object created with specified ID and complaint, empty patients list	Object created with ID=100, complaint= "Test Complaint", empty patients list
constructorWithNullPatientsList()	1	ID: 2222, Complaint: "Test Complaint 3", Patients list: null	NullPointerException thrown	NullPointerException thrown

setId(int id) & getId()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setId_valid()	1	ID: 54	ID updated to 54	ID updated to 54
setId_MIN()	1	ID: Integer.MIN_VALUE	ID updated to Integer.MIN_VALUE	ID updated to Integer.MIN_VALUE
setId_MAX()	1	ID: Integer.MAX_VALUE	ID updated to Integer.MAX_VALUE	ID updated to Integer.MAX_VALUE

setComplaint(String complaint) & getComplaint()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setComplaint_Valid()	1	Complaint: "Migraine"	Complaint updated to "Migraine"	Complaint updated to "Migraine"
testNullComplaintString()	1	Complaint: null	Complaint updated to null	Complaint updated to null

setPatientsList(List<Patients> patientsList) & getPatientsList()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setPatientsListValid()	1	List with 2 test patients	Patients list updated with 2 patients	Patients list updated with 2 patients
setPatientsList_NullList()	1	Patients list: null	NullPointerException thrown	NullPointerException thrown
setPatientsList_OriginalListUnmodified()	1	List 2 test patients, then clear complaint's list	Original list remains unchanged (2 patients), complaint's list is empty	Original list contains 2 patients, complaint's list is empty

testToString()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testToString()	1	Object with complaint: "Test Complaint"	"Test Complaint"	"Test Complaint"

fillComplaintList()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
complaintsListInitialization(	1	Object with complaint: "Test Complaint"	"Test Complaint"	"Test Complaint"
fillComplaintListNoDuplicates()	1	Call method multiple times	No duplicates added to complaint list	List size remains the same after multiple calls

getComplaintById(int id)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
getComplaintById_Valid()	1	ID: 2	Complaint object with ID= 2 & complaint= "Vomiting"	Complaint object with ID= 2 & complaint= "Vomiting"
getComplaintById_NonExistingComplain()	1	ID: 999	null	null
	2	ID: 0	null	null
	3	ID: -5	null	null

Relationship (Edge) Tests on Patients and Complaint				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
patientComplaintRelationship()	1	Create patient with complaint reference	Patient has complaint reference and complaint's patient list contains the patient	Patient has complaint reference and complaint's patient list contains the patient

multiplePatientsSameComplaint()	1	2 patients with same complaint	Complaint's patient list contains both patients	Complaint's patient list contains both patients
multipleComplaintBYOnePatient()	1	Patient with initial complaint changed to a new complaint	Patient's complaint updated to new complaint, patient remains in initial complaint's list, patient appears in new complaint's list	Patient's complaint updated to new complaint, patient remains in initial complaint's list, patient appears in new complaint's list

Class name - DoctorsTest.java

★ `title, branch, situation` - objects were created in `setup()` function with a Doctor object named `doctor.`

CONSTRUCTOR TESTS				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
defaultConstructor()	1	new Doctors()	Non-null Doctor object created	Non-null Doctor object
constructorWith_UserDetails()	1	"DR123456", "Adam", "Apple", title, branch, situation (all valid))	Doctor object with specified fields is created	Doctor object with specified fields is created
constructorWithoutUserDetails()	1	new Doctors(title, branch, situation)	Doctor object with specified fields (title, branch, situation) is created	Doctor object with specified fields (title, branch, situation) is created

★ Here in `defaultConstructor(),` a null Object should throw an error stating that a doctor without title branch can not be created but it creates a null object which is a bug that is not handled in code even with a pass test case.

setTitle(Titles title) & getTitle()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**

setTitle_NullInput()	1	null	IllegalArgumentException	✅ Pass
setTitle_Valid()	1	title (valid Titles object)	Title is set to provided value	✅ Pass

setBranch(Branches branch) & getBranch()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setBranch_Null()	1	null	IllegalArgumentException	✅ Pass
setBranch_Valid()	1	branch (valid Branches object)	Branch set correctly	Branch set correctly

setDoctorSituation(DoctorSituation doctorSituation) & getDoctorSituation()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setDoctorSituation_Null()	1	null	IllegalArgumentException	✅ Pass
setDoctorSituation_Valid()	1	situation (valid DoctorSituation object)	Doctor situation is set to the provided value	DoctorSituation set correctly

★ `Branch, DoctorSituation, Title`- are three concrete classes that the `Doctor` class was dependent on, thus their set methods were not tested thoroughly, only to the extent necessary to ensure that these class's objects will work perfectly in `Doctor`'s class.

addDoctor(Doctors doctor)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
addDoctor_Null()	1	null	IllegalArgumentException	✅ Pass
addDoctor_Valid(()	1	doctor (valid Doctors object), then doctor1 (another valid Doctors object)	Doctor is added to the list	Doctor added successfully

addDoctor_DuplicateId()	1	doctor1 (id="DR123"), then doctor2 (id="DR123", same id as doctor1)	IllegalArgumentException is thrown when adding doctor2.	✅ Pass

removeDoctor(String id)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
removeDoctor_ExistingId()	1	Crete Doctor Object and remove that Existing doctor ID	Doctor with the specified ID is removed from list, size decreased	Doctor removed, size = 0
removeDoctor_NonExistingId()	1	Non-existing doctor ID = "DR123"	IllegalArgumentException Thrown	✅ Pass

toString()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testToString()	1	Doctor object with all information	Formatted string with doctor details	Formatted string with doctor details

fillDoctorList()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
fillDoctorList_valid()	1	No input, **fillDoctorList()** called	doctorsList populated with 6 doctors	doctorsList populated with 6 doctors
fillDoctorList_DuplicacyTest()	1	No input, **fillDoctorList()** Called twice	No duplicate doctors added	List size unchanged, with no duplicacy

★ Below the `showUsers()` method is a part of superclass `User`; the purpose of this test case was to check whether it runs smoothly from `User` & whether it can show details of the `Doctor` object as part of `User`, an abstract superclass.

showUsers()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testShowUsers()	1	Doctor object with all information	Method executes without exceptions	Method executes without exceptions

getDoctorById(String id)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
getDoctorById_ ExistingId()	1	Existing doctor ID = "DR123"	Correct doctor object	Correct doctor object
getDoctorById_ NonExistingId()	1	"DR456" (non-existing ID)	IllegalArgumentExc eption	✅ Pass

getDoctorsList()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
getDoctorsList()	1	No input	List of all doctors	List containing all doctors
getDoctorsList_Cop yOfList()	1	No input	Original list remains unchanged (returns a copy)	Changes to returned list don't affect original

Inherited Method Test from Superclass				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setFirstName_Inh eritedMethod()	1	"Rifa"	Doctor’s first name set	doctor.firstname = “Rifa”
	2	"123"	IllegalArgumentException	✅ Pass
	3	""	IllegalArgumentException	✅ Pass
	4	null	IllegalArgumentException	✅ Pass

setLastName__InheritedMethod()	1	"Azad"	Doctor's Last name set	doctor.lastName = "Azad"
	2	"456"	IllegalArgumentException	✅ Pass
	3	""	IllegalArgumentException	✅ Pass
	4	null	IllegalArgumentException	✅ Pass

★ These methods were inherited from superclass **`User`**, thus only general validation were performed, without delving into edges test case checks which were done superclass.

Class name -PatientsTest.java

★ `birthDate, priority, complaint, lastStatus` - objects (parts of respective classes) were created in `setup()` function with a `Patients` object named `patient`.

CONSTRUCTOR TESTS				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
defaultConstructorTest()	1	No parameters	New Patient object created	New Patient object created
parameterizedConstructorTest()	1	"testId", "Lee", "Maha", "12345678901", birthDate, Gender.getGender(1), priority, complaint, lastStatus (all valid)	Patients object created with the provided values	Patient object with correct attributes
minimalParameterConstructorTest()	1	"12345678901", LocalDate.of(1990, 1, 15), Gender.getGender(1), priority, complaint, lastStatus	Patients object created	Patient object with minimal attributes

setTcNo(String tcNo) & getTcNo()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setTcNo_ValidInput()	1	"98765432109" (valid 11-digit	TC number set successfully	TC number "98765432109"

		number)		
setTcNo_NullInput()	1	null	IllegalArgumentException is thrown with message "Invalid TC number"	"Invalid TC number"
setTcNo_InvalidLength()	1	"12345" (invalid length Short)	IllegalArgumentException	✅ Pass
	2	"123456789012" (invalid length Short)	IllegalArgumentException	✅ Pass
setTcNo_InvalidChar()	1	"12345abcde" (invalid characters)	IllegalArgumentException	✅ Pass
	2	"12345 abcde"	IllegalArgumentException	✅ Pass
	3	“ “	IllegalArgumentException	✅ Pass

setBirthDate(LocalDate birthDate) & getBirthDate()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setBirthDate_ValidInput()	1	(1985, 5, 20) (a valid past date)	Patient's birth date is set successfully	✅ Pass
	2	(1800, 01, 01) Very old date	Birth date is set to 1800-01-01	✅ Pass (bug in code, should not take old dates)
setBirthDate_NullInput()	1	null	IllegalArgumentException is thrown with message "Invalid birth date.".	"Invalid birth date.".
setBirthDate_FutureDate()	1	LocalDate.now().plusDays(1) (a future date)	IllegalArgumentException is thrown with message "Invalid birth date.".	"Invalid birth date."
setBirthDate_PresentNowDate()	1	LocalDate.now() (today's date)	Patient's birth date is set successfully	✅ Pass

setGender(String gender) & getGender()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setGender_Valid()	1	"Female" (valid gender string)	Patient's gender is set to "Female".	✅ Pass
	2	“4566” (number)	Throw exception by not accepting anything other than complete string	✅ Pass (bug in code for not detecting string validation)
	3	“Hut75” (alphanemuric)		
setGender_NullInput()	1	null	NullPointerException is with the message "Gender cannot be null".	"Gender cannot be null".

★ `LastStatus, Priority, Complaint, Gender` - are four concrete classes thus their set methods were not tested thoroughly, just enough to know whether these class’s object will work perfectly in Patients’s class.

setPriority(Priorities priority) & getPriority()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testSetPriority()	1	priority (valid Priorities object)	Patient's priority is set to priority	✅ Pass
	2	null	priority set to NULLl	✅ Pass

setComplaint(Complaints complaint)) & getComplaint()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setComplaintTest()	1	complaint (valid Complaints object)	Patient's complaint is set to complaint	Complaint set successfully
	2	null	Complaint set to NULLl	✅ Pass

setLastStatus(LastStatuses lastStatus) & getLastStatus()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setLastStatus()	1	lastStatus (valid LastStatuses object)	Patient's last status is set to lastStatus.	Status set successfully
	2	null	LastStatus set to NULLl	✅ Pass

toString()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testToString()	1	Patient object with all attributes set	Returns a formatted string containing patient ID, first name, last name, TC No, birth date, gender, priority, complaint, and process status.	✅ Pass

fillPatientsList()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
fillPatientsListTest()	1	No input	List populated with 7 predefined patients	✅ Pass
noDuplicacyInMultipleFill()	1	Call fillPatientsList multiple times consecutively.	Patients.patientsList size remains the same after subsequent calls (no duplicates added)".	✅ Pass

getPatientById(String id)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
getPatientById_NonExistingId()	1	"NonExistentId" (non-existing patient ID)	null is returned.	null is returned.
	2	“ “	null is returned.	null is returned.

getPatientById_ExistingId()	1	"HT1001556" (existing patient ID from filled list)	Patients object is returned that is not null and whose first name is "Ali".	Successful return of object with that ID
getPatientById_NullId()	1	null	Is null is returned..	null is returned.

addPatient(Patients patient)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
addPatient_NonNull()	1	newPatient (a valid Patients object)	patientsList contains newPatient	Patient added to list
	2	Patient with same data but different ID: newPatient2	newPatient add successfully but newPatient2 gives Exception	✅ pass - meaning bug in code to not check information repetition
addPatient_NullPatient()	1	null	IllegalArgumentException is thrown with message "Patient cannot be null".	"Patient cannot be null".
addPatient_DuplicateId()	1	newPatient (id= "newId") then newPatient2 (id= "newId", same id as newPatient)	IllegalArgumentException is thrown when adding newPatient2	✅ Pass

removePatient(String id)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
removePatient_ExistingId()	1	"HT1001556" (existing patient ID)	Returns true, patient removed	✅ Pass
removePatient_NonExistingId()	1	"NonExistentId" (non-existing patient ID)	Returns false	✅ Pass

getAllPatients()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**

getAllPatientsTest()	1	No input	Returns a same size List with patientsList containing all patients	✅ Pass
getAllPatients_CopyOfList()	1	No input	Returns copy of patients list. Modifications to returned list (clearing an object) do not affect original	✅ Pass

showUsers()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
showUsers_PrintPatientList()	1	No input	No exception is thrown (verifies that the method can be called without errors, typically involves printing to console).	✅ Pass

Inherited Method Test from Superclass				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setFirstName_InheritedMethod()	1	"Rifa"	Patient First name set	patient.firstname = “Rifa”
	2	"123"	IllegalArgumentException	✅ Pass
	3	""	IllegalArgumentException	✅ Pass
	4	null	IllegalArgumentException	✅ Pass
setLastName__InheritedMethod()	1	"Azad"	Patient Last name set	patient.lastName = “Azad”
	2	"456"	IllegalArgumentException	✅ Pass
	3	""	IllegalArgumentException	✅ Pass
	4	null	IllegalArgumentException	✅ Pass

Class name - AppointmentTest.java

★ `title, branch, situation,doctor,patient, priority, lastStatus, appointDate, appointTime` - objects were created in `setup()` function with a Appointment object named `appointment` .

CONSTRUCTOR TESTS				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
constructorWithAutoGeneratedId()	1	patient, doctor, appointDate, appointTime (all valid)	Appointment created with auto-generated ID and correct attributes	Appointment object created correctly and accordingly
fullConstructor()	1	All fields provided ("APT-CUSTOM", dates, users, etc.)	Appointment created with exact specified attributes	
constructorSuperClass()	1	specificDate (LocalDateTime.of(2023, 5, 15, 10, 30))	Appointment created with specified creation date	

getCount() & setCount(int count)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testCountByCreatingAppointments()	1	Create two new Appointment instances after Appointment.setCount(0)	Appointment.getCount() returns 2.	✅ Pass
setCountTest()	1	setCount(initialCount + 5)	Appointment.getCount() returns initialCount + 5	✅ Pass

generateId()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testIdGeneration()	1	Create multiple appointments	IDs generated in sequence (APT-1, APT-2, etc.)	✅ Pass
testIdGenerationFormat()	1	Any new appointment	ID matches the pattern "APT-\d+".	✅ Pass

getPatient() & setPatient(Patients patient)				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setPatientValid()	1	patient (valid Patients object)	Patient set successfully	✅ Pass
setPatientNullException()	1	null	IllegalArgumentException is thrown	✅ Pass

setDoctor(Doctors doctor) & getDoctor()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setDoctorValid()	1	Doctor (valid Patients object)	Doctor set successfully	✅ Pass
setDoctor_Null()	1	null	Null accepted	returns null

setAppointmentDate(LocalDate appointmentDate) & getAppointmentDate()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setAppointmentDateValid()	1	LocalDate.now().plusDays(2) (valid future date)	Date set successfully	✅ Pass
	2	LocalDate.of(2077, 03, 12), far future date	Date is set to provided value	✅ Pass
setAppointmentDateNullException()	1	null	IllegalArgumentException is thrown	✅ Pass
setAppointmentDateToday()	1	LocalDate.now() (today's date)	Future Date set successfully	✅ Pass
setAppointmentDate_PastDateException()	1	LocalDate.now().minusDays(1) (past date)	Past Date set successfully	✅ Pass

setAppointmentTime(LocalTime appointmentTime) & getAppointmentTime()

Function Name	Test Case ID	Input	Expected Output	Actual Output
setAppointmentTimeValid()	1	LocalTime.of(11, 0) (valid time within business hours)	Appointment Time set successfully	✅ Pass
	2	Exactly 9:00 AM (start of business hours)	Time is set to 9:00 AM	✅ Pass
	3	Exactly 17:00 (5:00 PM - end of business hours)	Time is set to 17:00	✅ Pass
setAppointmentTime_RejectNUll()	1	null	IllegalArgumentException is thrown	✅ Pass
setAppointmentTimeOutsideBusinessHoursEarlyException()	1	LocalTime.of(8, 59) (time before 9 AM)	IllegalArgumentException is thrown	✅ Pass
setAppointmentTimeOutsideBusinessHoursLateException()	1	LocalTime.of(17, 1) (time after 5 PM)	IllegalArgumentException is thrown	✅ Pass

setStatus(AppointmentStatus status) & getStatus()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
setStatusTest()	1	AppointmentStatus.CONFIRMED (or any other value from the enum)	Status updated successfully	✅ Pass
getStatusReturnsDefaultEnumStatus()	1	No input	Default status is SCHEDULED	returns null

toString()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testToString()	1	No input	Returns a format that contains all relevant appointment info	✅ Pass

★ Below three methods (edge cases) were tested keeping the real world system in mind. In a real application, these validation would typically be implemented in a service or repository layer. But the current implementation doesn't prevent double-booking, as we tested that both appointments can be created with the same doctor at the same time by 2 different patients, also same patient booking multiple appointments at the same time. In a real system, this would be prevented. Below are the implementations for these scenarios.

Conflict Check(Patient, Doctor), Multiple Appointments,				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testMultipleAppointmentsWithSameDoctorDifferentTime()	1	Two appointments with the same doctor, same date, but different times	Appointments set successfully	✅ Pass
testMultipleAppointmentsWithSameDoctorDifferentTime()	1	Two appointments with different patients, same doctor, same date, same time	Appointments set successfully	✅ Pass (should not have passed)
testMultipleAppointmentsWithSameDoctorDifferentTime()	1	Two appointments with the same patient, different doctors, same date, same time.	Appointments set successfully	✅ Pass (should not have passed)

Class name - UsersTest.java

getId()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testSetterWithValidId()	1	"DR500111"	"DR500111"	"DR500111"
testRepeatedSetters()	1	"DR500111","DR502333", "DR504556"	"DR504556"	"DR504556"
testValidUserCreation()	1	"id123", "John", "Doe"	"id123", "John", "Doe"	"id123", "John", "Doe"

testUpdateToNewId()	1	“HT1000012”, “Tripty”, “Hossain”, Update with: “HT1000013”	Updated to “HT1000013”	Updated to “HT1000013”

setId()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testSetIdNull()	1	Null	Throws IllegalArgument Exception	Throws IllegalArgumentException
testSetIdEmpty()	1	“ ”	Throws IllegalArgument Exception	Throws IllegalArgumentException
testSetIdSpecialCharacter()	1	"id$123"	Throws IllegalArgument Exception	Throws IllegalArgumentException
testSetIdWithSpace()	1	"HT10 015 56"	Throws IllegalArgument Exception	Throws IllegalArgumentException
testSetIdCaseSensitive()	1	"ht1000001"	Not Equal to "HT1000001"	PASS
testSetIdToDuplicate()	1	User1:“HT1000008”, “Tripty”, “Hossain” User2:“HT1000008”, “Tahniyat”, “Tuli”	Throws IllegalArgument Exception	Throws IllegalArgumentException

getFirstName()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testValidFirstName()	1	"Tripty"	"Tripty"	"Tripty"

testRepeatedSettersFirstName()	1	"Tripty", "Tasnova", "Rifa"	"Rifa"	"Rifa"
testFirstNameCaseSensitive()	1	"tripty"	Not Equal to "TRIPTY"	PASS
testUpdateToNewFirstName()	1	“HT1000111”, “Tripty”, “Hossain”, Update with: “Ety”	Updated to “Ety”	Updated to “Ety”

setFirstName()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testSetFirstNameNull()	1	Null	Throws IllegalArgumentException	Throws IllegalArgumentException
testSetFirstNameEmpty()	1	“ ”	Throws IllegalArgumentException	Throws IllegalArgumentException
testSetFirstNameWithSpecialCharacter()	1	"Tr!pty"	Throws IllegalArgumentException	Throws IllegalArgumentException
testSetFirstNameWithDigit()	1	"Tripty123"	Throws IllegalArgumentException	Throws IllegalArgumentException
testSetFirstNameWithSpace()	1	"Tri pty "	Throws IllegalArgumentException	Throws IllegalArgumentException
testDuplicateFirstNameForDifferentId()	1	User1:“HT1000015”, “Tripty”, “Hossain” User2:“HT1000015”, “Tripty”, “Hossain”	Allowed	PASS

getLastName()				
Function Name	**Test**	**Input**	**Expected**	**Actual**

	Case ID		Output	Output
testValidLastName()	1	"Hossain"	"Hossain"	"Hossain"
testRepeatedSettersLastName()	1	"Akter","Begum", "Hossain"	"Hossain"	"Hossain"
testLastNameCaseSensitive()	1	"hossain"	Not Equal to "Hossain"	PASS
testUpdateToNewLastName()	1	“HT1000011”, “Tripty”, “Hossain”, Update with: “Begum”	Updated to: “Begum”	Updated to: “Begum”

setLastName()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testSetLastNameNull()	1	Null	Throws IllegalArgument Exception	Throws IllegalArgumentException
testSetLastNameEmpty()	1	“ ”	Throws IllegalArgument Exception	Throws IllegalArgumentException
testSetLastNameWithSpecialCharacter()	1	"H@ssa!n"	Throws IllegalArgument Exception	Throws IllegalArgumentException
testSetLastNameWithDigit()	1	"Hossain456"	Throws IllegalArgument Exception	Throws IllegalArgumentException
testSetLastNameWithSpace()	1	"Hos sain "	Throws IllegalArgument Exception	Throws IllegalArgumentException
testDuplicateLastNameForDifferentId()	1	User1:“HT1000001”, “Tripty”, “Hossain” User2:“HT1000002”, “Tripty”, “Hossain”	Allowed	PASS

showUsers()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testShowUsersDoesNotThrow()	1	Call show user with “HT1004111”, “Asli”, “Yilmaz”	Method runs without Exception	Method runs without Exception

Class name -StartTest.java

start()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testExitOption()	1	0	Exit	Exit
testInvalidIntegerInput()	1	5	“Invalid input !”	“Invalid input !”
testNonIntegerInput()	1	0.00	ThrowsInputMismatchException	ThrowsInputMismatchException
testNonDigitInput()	1	abc	ThrowsInputMismatchException	ThrowsInputMismatchException
testNegativeInput()	1	-1	“Invalid input !”	“Invalid input !”
testLargeInputNumber()	1	9999999	“Invalid input !”	“Invalid input !”
testSpecialCharacterInput()	1	@	ThrowsInputMismatchException	ThrowsInputMismatchException
testInputNull()	1	null	ThrowsInputMismatchException	ThrowsInputMismatchException

Class name - AmbulanceScheduleValidatorTest.java

getValidPickupDate()				
Function Name	**Test Case Id**	**Input**	**Expected Output**	**Actual Output**
testValidPickupDate()	1	Localdate.now().plusDays(2)	02-05-2025	02-05-2025
testPastPickupDate()	1	Localdate.now().minusDays(2)	False	False
testInvalidDayPickupInput()	1	31,4,2025	Throws Exception	Throws Exception
testInvalidMonthPickupInput()	1	13,13,2025	Throws Exception	Throws Exception
testNonNumericPickupInput()	1	abc	ThrowInputMismatchException	ThrowInputMismatchException
testNegativePickupDate()	1	-13,11,2025	Throws Exception	Throws Exception
testLeapYearPickupDate()	1	29,2,2028	29-02-2028	29-02-2028
testInvalidLeapYearPickupDate()	1	29,2,2026	Throws Exception	Throws Exception

getValidReturnDate()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testValidReturnDate()	1	Localdate.now().plusDays(2) Localdate.now().plusDays(4)	True	True
testInvalidReturnDate()	1	Localdate.now().plusDays(3)	False	False

		Localdate.now().plusDays(2)		
samePickupReturnDate()	1	Localdate.now().plusDays(2) Localdate.now().plusDays(2)	True	True
testInvalidDayReturnDate()	1	Localdate.now().plusDays(2) 31,6,2025	Throws Exception	Throws Exception
testInvalidMonthReturnDate()	1	Localdate.now().plusDays(2) 11,14,2025	Throws Exception	Throws Exception
testNonNumericReturnDate()	1	Localdate.now().plusDays(2) abc	ThrowInputMismatchException	ThrowInputMismatchException
testNegativeReturnDate()	1	Localdate.now().plusDays(2) -11,12,2025	Throws Exception	Throws Exception
testLeapYearReturnDate()	1	Localdate.now().plusDays(2) 29,2,2028	True	True
testInvalidLeapYearReturnDate()	1	Localdate.now().plusDays(2) 29,2,2026	Throws Exception	Throws Exception

Class name : TimeValidatorTest.java

getValidTime()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testValidInput()	1	10:30	10:30	10:30
testInvalidInput()	1	abc	ThrowsInputMismatchException	ThrowsInputMismatchException

testZeroHourZeroMinute()	1	00:00	00:00	00:00
testUpperBoundaryTime()	1	23:59	23:59	23:59
testOutOfBoundHour()	1	25:15	Throws Exception	Throws Exception
testOutOfBoundMinute()	1	14:61	Throws Exception	Throws Exception
testNegativeHour()	1	-5:30	Throws Exception	Throws Exception
testNegativeMinute()	1	10:-15	Throws Exception	Throws Exception

Class name :TcNoValidatorTest.java

getValidTcNumber()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testValidTcNumber()	1	01734567890	01734567890	01734567890
testNonDigitInput()	1	abc123def	False	False
testShortTcNumber()	1	017345678	False	False
testLongTcNumber()	1	0173456789012345	False	False
testNegativeInput()	1	0-1734567890	False	False
testSpecialCharacterInput()	1	1234@5678!	False	False
testNullInput()	1	Null	Null	Null
testWhiteSpaceTcNumber()	1	0173 7890	False	False

Class name - NameValidatorTest

isValidFirstName()				
Function Name	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
isValidFirstName	1	input(“Tasnuva”)	Tasnuva	Tasnuva
isValidFirstName_ allLowerCase	1	input(“tasnuva”)	Tasnuva	Tasnuva
isValidFirstName_ middleUpperCase	1	input(“tasNuva”)	TasNuva	TasNuva
isValidFirstName_ multipleUpperCase	1	input(“TAsNuva”)	TAsNuva	TAsNuva
isValidFirstName_ shortLength	1	input(“T”)	null	null
isValidFirstName_ largeLength	1	input(“Tasnuva Islam Milaaaaaaa”)	null	null
isValidFirstName_ WithEmptyInput	1	input(“”)	null	null
isValidFirstName_ WithOnlySpaces	1	input(“ ”)	null	null
isValidFirstName_ startWithSpaces	1	input(“ tasnuva”)	Tasnuva	Tasnuva
isValidFirstName_ endWithSpaces	1	input(“tasnuva ”)	Tasnuva	Tasnuva
isValidFirstName_ startAndEndWithSpaces	1	input(“ tasnuva ”)	Tasnuva	Tasnuva
isValidFirstName_ endWithNumbers	1	input(“Tasnuva033”)	null	null
isValidFirstName_	1	input(“2022Tasnuva”)	null	null

startWithNumbers				
isValidFirstName_ startAndEndWithNumbers	1	input(“324Tasnuva345”)	null	null
isValidFirstName_ mixedNumbers	1	input(“324Tas8nu345”)	null	null
isValidFirstName WithSpecialInput	1	input("Tasnu%@Islam")	null	null
	2	input("Tasnuva-Islam")	null	null
	3	input("$Tasnuva")	null	null

isValidLastName()				
Function Name	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
isValidLastName	1	input(“Islam”)	Islam	Islam
isValidLastName_ allLowerCase	1	input(“islam”)	Islam	Islam
isValidLastName_ middleUpperCase	1	input(“isLam”)	IsLam	IsLam
isValidLastName_ multipleUpperCase	1	input(“IsLaM”)	IsLaM	IsLaM
isValidLastName_ shortLength	1	input(“I”)	null	null
isValidLastName_ largeLength	1	input(“Tasnuva Islam Milaaaaaaa”)	null	null
isValidLastName_ WithEmptyInput	1	input(“”)	null	null
isValidLastName_ WithOnlySpaces	1	input(“ ”)	null	null
isValidLastName_ startWithSpaces	1	input(“ islam”)	Islam	Islam

isValidLastName_ endWithSpaces	1	input(“islam ”)	Islam	Islam
isValidLastName_ startAndEndWithSpaces	1	input(“ islam ”)	Islam	Islam
isValidLastName_ endWithNumbers	1	input(“Islam033”)	null	null
isValidLastName_ startWithNumbers	1	input(“2022Islam”)	null	null
isValidLastName_ startAndEndWithNumbers	1	input(“324Islam345”)	null	null
isValidLastName_ mixedNumbers	1	input(“324Isl68am345”)	null	null
isValidLastName_ SpecialChars	1	input("Tasnu%@Islam")	null	null
	2	input("Tasnuva-Islam")	null	null
	3	input("$Islam")	null	null

Class name - GenderTest

getGender()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
male	1	getGender(1)	Male	Male
female	1	getGender(2)	Female	Female
other	1	getGender(6)	other	other
zero	1	getGender(0)	other	other

negative	1	getGender(-1)	other	other
largePositive	1	getGender(1000)	other	other

Class name - DoctorSituationTest

getId()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getId	1	DoctorSituation(20, “On leave”)	20	20
getId_ notEquals	1	DoctorSituation(21, “In Surgery”)	20	21

getSituation()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getSituation	1	DoctorSituation(20, “On leave”)	On leave	On leave
getSituation_ notEquals	1	DoctorSituation(21, “In Surgery”)	On leave	In Surgery

fillSituation()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
fillSituation_ listSize	1	situationList.size()	4	4
fillSituation_ getFirstIndexID	1	situationList.get(0) .getId()	20	20
fillSituation_ getFirstIndexSit	1	situationList.get(0) .getSituation()	On leave	On leave

uation				
fillSituation_ getThirdIndexSituation	1	situationList.get(2) .getSituation()	Available	Available
fillSituation_ getForthIndexID	1	situationList.get(3) .getId()	23	23
fillSituation_ getForthIndexSituation	1	situationList.get(3) .getSituation()	In Examination	In Examination
fillSituation_ getIdNotEquals	1	situationList.get(1) .getId()	22	20
fillSituation_ getSituationNotEquals	1	situationList.get(1) .getSituation()	In Examination	On leave

getSituationById()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getSituationById	1	getSituationById(21)	In Surgery	In Surgery
getSituationById_outOfList	1	getSituationById(30)	null	null
getSituationById_negativeId	1	getSituationById(-1)	null	null

Class name - LastStatusesTest

getId()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**

getId	1	lastStatuses(1, “on going”)	1	1
getId_notEquals	1	lastStatuses(2, “completed”)	1	2

getLastStatus()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getLastStatus	1	lastStatuses(3, “discharge”)	discharge	discharge
getLastStatus_ notEquals	1	lastStatuses(2, “completed”)	on going	completed

fillLastStatusList()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
fillLastStatusList_ getListSize	1	lastStatusesList .size()	3	3
fillLastStatusList_ getFirstIndexSituat ion	1	lastStatusesList .get(0).getStatus()	on going	on going
	2	lastStatusesList .get(0).getStatus()	actual	on going

getLastStatusById()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getLastStatusById	1	getLastStatusById(1)	on going	on going
getLastStatusById_ outOfList	1	getLastStatusById(4)	null	null
getLastStatusById_	1	getLastStatusById(0)	null	null

zero				
getLastStatusById_ negativeIndex	1	getLastStatusById(-1)	null	null

Class name - PrioritiesTest

getId()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getId	1	Priorities(34, “urgent”)	34	34
getId_notEquals	1	Priorities(34, “urgent”)	40	34

getPriority()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getPriority	1	Priorities(33, “Routine”)	Routine	Routine
getPriority_ notEquals	1	Priorities(33, “Routine”)	Emergency	Routine

fillPrioritiesList()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
fillPrioritiesList_ getListSize	1	prioritiesList.size()	3	3
fillPrioritiesList_ firstIndexPriority	1	prioritiesList.get(0) .getPriority()	Emergency	Emergency

fillPrioritiesList_ notEquals	1	prioritiesList.get(2) .getPriority()	Outpatient Clinic	Routine

getPrioritiesById()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getPrioritiesById	1	getPrioritiesById(32)	Outpatient Clinic	Outpatient Clinic
getPrioritiesById_ outOfIndex	1	getPrioritiesById(34)	null	null
getPrioritiesById_ zeroIndex	1	getPrioritiesById(0)	null	null
getPrioritiesById_ negativeIndex	1	getPrioritiesById(-1)	null	null

Class name - TitlesTest

getId()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getId	1	Titles(11, “Specialist”)	11	11
getId_notEquals	1	Titles(11, “Specialist”)	14	11

getTitle()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getTitle	1	Titles(15, “Professor”)	Professor	Professor
getTitle_	1	Titles(12, “Operator”)	Associate	Operator

notEquals			Professor	

fillTitles()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
fillTitles_getListSize	1	titlesList.size()	6	6
fillTitles_getFirstIndexTitle	1	titlesList.get(0).getTitle()	Practitioner	Practitioner
fillTitles_getFirstIndexTitleNotEquals	1	titlesList.get(0).getTitle()	Associate Professor	Routine

getTitleById()				
Name of test function	**Test Case Id**	**Input**	**Expected output**	**Actual Output**
getTitleById	1	getTitleById(13)	Associate Professor	Associate Professor
getTitleById_outOfIndex	1	getTitleById(20)	null	null
getTitleById_zeroIndex	1	getTitleById(0)	null	null
getTitleById_negativeIndex	1	getTitleById(-1)	null	null

Class name - AppointmentManagerTest.java

Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**

testIdMaker_withValidNumber	1	456	APT-456	APT-456
testIdMaker_withNullNumber	1	null	INVALID-ID	INVALID-ID
testIdMaker_withEmptyString	1	“ “	INVALID-ID	INVALID-ID
testIdMaker_withInvalidCharacters	1	abc123	INVALID-ID	INVALID-ID
testIdMaker_withSpecialCharacters	1	12#45	INVALID-ID	INVALID-ID

Class name -DoctorManagerTest

idmaker()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
idMaker_WithShortNumber	1	“123”	"DR558123"	"DR558123"
idMaker_WithLongNumber	1	"123478901"	"DR559901"	"DR559901"
idMaker_LessThanThreeDigits	1	"1"	Throws Exception	Throws Exception
idMaker_NullInput	1	null	Throws Exception	Throws Exception
idMaker_EmptyString	1	" "	Throws Exception	Throws Exception

Class name -PatientManagerTest

idmaker()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
idMaker_shouldGenerateCorrectIdWithLongNumber	1	"12345678901"	"HT1006901"	"HT1006901"
idMaker_shouldGenerateCorrectIdWithShortNumber	1	“123”	"HT1007123"	"HT1007123"
idMaker_shouldThrowExceptionForLessThanThreeDigits	1	"1"	Throws Exception	Throws Exception
idMaker_shouldHandleNullInputGracefully	1	null	Throws Exception	Throws Exception
idMaker_shouldHandleEmptyString	1	" "	Throws Exception	Throws Exception

Class name -DateValidatorTest

getValidDate()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testValidDateInput	1	15\n4\n2023\n	2023-04-15	2023-04-15
testInvalidThenValidDateInput	1	abc\nxyz\n123\n15\n4\n2023\n	2023-04-15	Throws Exception
testInvalidDateThenValid	1	31\n2\n2023\n15\n3\n2023\n	2023-03-15	Throws Exception
testLeapYearValidDate	1	29\n2\n2024\n	2024-02-29	2024-02-29

getValidDate()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testLeapYearInvalidDateThenValid	1	29\n2\n2023\n28\n2\n2023\n	2023-02-28	2023-02-28
testInvalidDateFormat	1	abc\nxyz\n!@#\n	InputMismatchException	Throws Exception

Class name -IdMakerTest

idMaker()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
idMaker_shouldPrefixWithID	1	"456"	"ID-456"	"ID-456"
idMaker_shouldWorkWithEmptyString	1	" "	"ID-"	"ID-"
idMaker_shouldWorkWithAlphabets	1	"ABC"	"ID-ABC"	"ID-ABC"
idMaker_shouldThrowForSpecialCharacters	1	"#"	IllegalArgumentException	IllegalArgumentException
idMaker_shouldWorkWithSpaces	1	" 123 "	"ID- 123 "	"ID- 123 "
idMaker_shouldThrowNullPointerExceptionForNullInput	1	null	NullPointerException	NullPointerException

idMaker()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
idMaker_shouldWorkWithLongInput	1	"7648388383474389838 47438"	"ID-764838838347 438983847438"	"ID-7648388383474 38983847438"
idMaker_shouldBeCaseSensitive	1	"abc" and "ABC"	Should not be equal	Not equal

Class name -InputValidatorTest

getInput()				
Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testValidNumericInput	1	"123"	"123"	"123"
testInvalidNonNumericInput	1	"abc\n123\n"	"123"	"123"
testEmptyInput	1	"\n123\n"	"123"	"123"
testInputWithSpaces	1	" \n123\n"	"123"	"123"
testMixedInput	1	"12abc34\n567\n"	"567"	"1234"
testMultipleInvalidInputs	1	"abc\ndef\n123\n"	"123"	"123"
testValidInputAfterEmptyAndInvalidInputs	1	"\nabc\n\n456\n"	"456"	"456"

Class name -UtilitiesTest

Function Name	**Test Case ID**	**Input**	**Expected Output**	**Actual Output**
testValidDoctorId_D001	1	"D001"	["On Duty", "Available"]	["On Duty", "Available"]
testValidDoctorId_D002	1	"D002"	["In Surgery", "Busy"]	["In Surgery", "Busy"]
testDoctorWithSingleSituation	1	"D003"	["On Leave"]	["On Leave"]
testUnknownDoctorIdreturnsEmptyList	1	"D999"	[] (empty list)	[] (empty list)
testEmptyDoctorIdreturnsEmptyList	1	" "	[] (empty list)	[] (empty list)
testNullDoctorIdthrowsException	1	null	IllegalArgumentException	IllegalArgumentException
testDoctorIdIsCaseSensitive	1	"d001"	[] (empty list, since key is case-sensitive)	[] (empty list)

Contribution Table

Name	ID	Contribution part
Rifa Azad	2022-1-60-265	BranchesTest.java (29 Test functions) EntityBaseTest.java (30 Test functions) DoctorsTest.java (24 Test functions) PatientsTest.java (32 Test functions) ComplaintsTest.java (20 Test functions) AppointmentTest.java (25 Test functions)
Tahura Akter Tripty	2022-1-60-051	UserTest.java (31 Test cases) StartTest.java (08 Test cases) AmbulanceScheduleValidatorTest.java (17 Test cases) TimeValidatorTest.java (08 Test cases) TcNoValidatorTest.java (08 Test cases)
Shaiful Islam	2022-1-60-120	DateValidatorTest.java (6 Test Cases) IdMakerTest.java (8 Test Cases) InputValidatorTest.java (7 Test Cases) UtilitiesTest.java (7 Test Cases)
Umme Habiba Rupu	2022-1-60-334	AppointmentManagerTest.java(5 Test Cases) DoctorManagerTest.java(5 Test Cases) PatientManagerTest.java(5 Test Cases) PatientMenuManagerTest.java(not valid for testing) DoctorMenuManagerTest.java AppointmentMenuManager.java
Tasnova Islam Mila	2022-1-60-033	NameValidatorTest.java (36 Test functions) GenderTest.java (6 Test functions) DoctorSituationTest.java (15 Test functions) LastStatusesTest.java (11 Test functions) PrioritiesTest.java (11 Test functions) TitlesTest.java (12 Test functions)